Machine-Level Data Representation	1		
Assembly Level Machine Organization	1		1 + 1 (PDC)
Memory Hierarchy	4+2 (OS)		
Interfacing and Communication	1		
Functional Organization			2
Performance and Energy Efficiency			3
Heterogeneous Architectures			2
Secure Processor Architectures			2
Quantum Architectures			2
Sustainability Issues	Included in SEP hours		
Total	**9**		**16**

The hours shared with OS include overlapping topics and are counted here.

Knowledge Units

AR-Logic: Digital Logic and Digital Systems

KA Core:

1. Combinational vs sequential logic/field programmable gate arrays (FPGAs) (See also: SF-Overview, SF-Foundations, SPD-Embedded)
   a. Fundamental combinational
   b. Sequential logic building block
2. Computer-aided design tools that process hardware and architectural representations
3. High-level synthesis
   a. Register transfer notation
   b. Hardware description language (e.g., Verilog/VHDL/Chisel)
4. System-on-chip (SoC) design flow
5. Physical constraints
   a. Gate delays
   b. Fan-in and fan-out
   c. Energy/power
   d. Speed of light

Illustrative Learning Outcomes:

KA Core:

1. Discuss the progression of computer technology components from vacuum tubes to VLSI, from mainframe computer architectures to the organization of warehouse-scale computers.
2. Describe parallelism and data dependencies between and within components in a modern heterogeneous computer architecture.
3. Explain the relationship between parallelism and power consumption.
4. Construct the design of basic building blocks for a computer: arithmetic-logic unit (gate-level), registers (gate-level), central processing unit (register transfer-level), and memory (register transfer-level).
5. Evaluate simple building blocks (e.g., arithmetic-logic unit, registers, movement between registers) of a simple computer design.
6. Analyze the timing behavior of a pipelined processor, identifying data dependency issues.

AR-Representation: Machine-Level Data Representation

CS Core:

1. Overview and history of computer architecture (See also: SPD-Game)
2. Bits, bytes, and words
3. Unsigned, signed and two's complement representations
4. Numeric data representation and number bases
   a. Fixed-point
   b. Floating-point
5. Representation of non-numeric data
6. Representation of records, arrays and UTF data types (See also: AL-Foundational)

Illustrative Learning Outcomes:

CS Core:

1. Discuss why everything in computers are data, including instructions.
2. Explain how fixed-length number representations can affect accuracy and precision.
3. Describe how negative integers are stored in sign-magnitude and two's-complement representations.
4. Discuss how different formats can represent numerical data.
5. Explain the bit-level representation of non-numeric data, such as characters, strings, records, and arrays.
6. Translate numerical data from one format to another.
7. Describe how a single adder (without overflow detection) can handle both signed (two's complement) and unsigned (binary) input without "knowing" which format a given input is using.

AR-Assembly: Assembly Level Machine Organization

CS Core:

1. von Neumann machine architecture
2. Control unit: instruction fetch, decode, and execution (See also: OS-Principles)
3. Introduction to SIMD vs MIMD and the Flynn taxonomy (See also: PDC-Programs, OS-Scheduling, OS-Process)

4. Shared memory multiprocessors/multicore organization (See also: PDC-Programs, OS-Scheduling)

KA Core:

5. Instruction set architecture (ISA) (e.g., x86, ARM and RISC-V)
   a. Fixed vs variable-width instruction sets
   b. Instruction formats
   c. Data manipulation, control, I/O
   d. Addressing modes
   e. Machine language programming
   f. Assembly language programming
6. Subroutine call and return mechanisms (See also: FPL-Translation, OS-Principles)
7. I/O and interrupts (See also: OS-Principles)
8. Heap, static, stack, and code segments (See also: FPL-Translation, OS-Process)

Illustrative Learning Outcomes:

CS Core:

1. Discuss how the classical von Neumann functional units are implemented in embedded systems, particularly on-chip and off-chip memory.
2. Describe how instructions are executed in a classical von Neumann machine, with extensions for threads, multiprocessor synchronization, and SIMD execution.
3. Assess an example diagram with instruction-level parallelism and hazards to describe how they are managed in typical processor pipelines.

KA Core:

4. Discuss how instructions are represented at the machine level and in the context of a symbolic assembler.
5. Map an example of high-level language patterns into assembly/machine language notations.
6. Contrast different instruction formats considering aspects such as addresses per instruction and variable-length vs fixed-length formats.
7. Analyze a subroutine diagram to comment on how subroutine calls are handled at the assembly level.
8. Describe basic concepts of interrupts and I/O operations.
9. Write a simple assembly language program for string/array processing and manipulation.

AR-Memory: Memory Hierarchy

CS Core:

1. Memory hierarchy: the importance of temporal and spatial locality (See also: SF-Performance, OS-Memory)
2. Main memory organization and operations (See also: OS-Memory)
3. Persistent memory (e.g., SSD, standard disks)
4. Latency, cycle time, bandwidth, and interleaving (See also: SF-Performance)
5. Cache memories (See also: SF-Performance)
   a. Address mapping
   b. Block size

    c. Replacement and store policy
    d. Prefetching
6. Multiprocessor cache coherence (See also: OS-Scheduling)
7. Virtual memory (hardware support) (See also: OS-Memory)
8. Fault handling and reliability (See also: SF-Reliability)
9. Reliability (See also: SF-Reliability, OS-Faults)
    a. Error coding
    b. Data compression
    c. Data integrity

KA Core:

10. Processing In-Memory (PIM)

Illustrative Learning Outcomes:

CS Core:

1. Using a memory system diagram, identify the main types of memory technology (e.g., SRAM, DRAM) and their relative cost and performance.
2. Measure the effect of memory latency on running time.
3. Enumerate the functions of a system with virtual memory management.
4. Compute average memory access time under various cache and memory configurations and mixes of instruction and data references.

AR-IO: Interfacing and Communication

CS Core:

1. I/O fundamentals (See also: OS-Devices, PDC-Communication)
    a. Handshaking and buffering
    b. Programmed I/O
    c. Interrupt-driven I/O (See also: OS-Principles)
2. Interrupt structures: vectored and prioritized, interrupt acknowledgment (See also: OS-Principles)
3. I/O devices (e.g., mouse, keyboard, display, camera, sensors, actuators) (See also: GIT-Fundamentals, GIT-Interaction, OS-Advanced-Files, PDC-Programs)
4. External storage, physical organization, and drives
5. Buses fundamentals (See also: OS-Devices)
    a. Bus protocols
    b. Arbitration
    c. Direct-memory access (DMA)

Illustrative Learning Outcomes:

CS Core:

1. Analyze an interrupt control diagram to comment on how interrupts are used to implement I/O control and data transfers.
2. Enumerate various types of buses in a computer system.
3. List the advantages of magnetic disks and contrast them with those of solid-state disks.

AR-Organization: Functional Organization

KA Core:

1. Implementation of simple datapaths, including instruction pipelining, hazard detection, and resolution (e.g., stalls, forwarding)
2. Control unit
   a. Hardwired implementation
   b. Microprogrammed realization
3. Instruction pipelining (See also: SF-Overview)
4. Introduction to instruction-level parallelism (ILP) (See also: PDC-Programs)

Illustrative Learning Outcomes:

KA Core:

1. Compare alternative implementation of datapaths in modern computer architectures.
2. Produce a set of control signals for adding two integers using hardwired and microprogrammed implementations.
3. Discuss instruction-level parallelism using pipelining and significant hazards that may occur.
4. Design a complete processor, including datapath and control.
5. Compute the average cycles per instruction for a given processor and memory system implementation.

AR-Performance-Energy: Performance and Energy Efficiency

KA Core:

1. Performance-energy evaluation (introduction): performance, power consumption, memory, and communication costs (See also: SF-Evaluation, OS-Scheduling, SPD-Game)
2. Branch prediction, speculative execution, out-of-order execution, Tomasulo's algorithm
3. Enhancements for vector processors and GPUs (See also: SPD-Game)
4. Hardware support for multithreading (See also: OS-Concurrency, OS-Scheduling, PDC-Programs)
   a. Race conditions
   b. Lock implementations
   c. Point-to-point synchronization
   d. Barrier implementation
5. Scalability
6. Alternative architectures including VLIW/EPIC, accelerators, and other special purpose processors
7. Dynamic voltage and frequency scaling (DVFS)
8. Dark Silicon

Illustrative Learning Outcomes:

KA Core:

1. Discuss performance and energy efficiency evaluation metrics.
2. Analyze a speculative execution diagram and write about the decisions that can be made.
3. Create a GPU performance-watt benchmarking diagram.
4. Write a multithreaded program that adds (in parallel) elements of two integer vectors.
5. Recommend a set of design choices for alternative computer architectures.

6. Enumerate key concepts associated with dynamic voltage and frequency scaling.
7. Measure energy savings improvement for an 8-bit integer quantization compared to a 32-bit quantization.

AR-Heterogeneity: Heterogeneous Architectures

KA Core:

1. SIMD and MIMD architectures (e.g., General-Purpose GPUs, TPUs, and NPUs) (See also: PDC-Programs, SPD-Embedded, GIT-Shading, SPD-Game)
2. Heterogeneous memory systems (See also: OS-Process, PDC-Communication)
   a. Shared memory versus distributed memory
   b. Volatile vs non-volatile memory
   c. Coherence protocols
3. Domain-Specific Architectures (DSAs) (See also: HCI-Accountability, GIT-Shading)
   a. Machine Learning Accelerator
   b. In-networking computing (See also: NC-Applications)
   c. Embedded systems for emerging applications
   d. Neuromorphic computing
   e. Edge computing devices
4. Packaging and integration solutions such as 3DIC and chiplets
5. Machine learning in architecture design
   a. AI algorithms for workload analysis
   b. Optimization of architecture configurations for performance and power efficiency

Illustrative Learning Outcomes:

KA Core

1. Analyze a system diagram with alternative parallel architectures, e.g., SIMD and MIMD, and identify the key differences.
2. Discuss what memory-management issues are found in multiprocessors that are not present in uniprocessors and how these issues might be resolved.
3. Indicate the differences between memory backplane, processor memory interconnect, and remote memory via networks, their implications for access latency, and their impact on program performance.
4. Discuss how you would determine when to use a domain-specific accelerator instead of a general-purpose CPU.
5. Enumerate key differences in architectural design principles between a vector and scalar-based processing unit.
6. List the advantages and disadvantages of a PIM architecture.

AR-Security: Secure Processor Architectures

KA core:

1. Principles of Secure Hardware
   a. Security Risk Analysis, Asset Protection, and Threat Model
   b. Cryptographic Acceleration with Hardware (See also: SEC-Crypto)

   c. Support for virtualization (e.g., OS isolation)
2. Roots of trust in hardware, Physically Unclonable Functions (PUF)
3. Hardware Random Number Generators
4. Memory protection extensions
   a. Runtime pointer bounds checking (e.g., buffer overflow)
   b. Protection at the microarchitectural level
   c. Protection at the ISA level
5. Trusted Execution Environment (TEE)
   a. Trusted Computer Base Protections
   b. Protecting virtual machines
   c. Protecting containers
   d. Trusted software modules (Enclaves)
6. Homomorphic encryption for privacy-preserving data processing

Illustrative Learning Outcomes

KA Core:

1. Discuss principles of secure hardware, exploring a framework for risk analysis and asset protection.
2. Summarize how Physically Unclonable Functions (PUF) can be a unique device identifier in security applications.
3. Distinguish a random number generator with dedicated hardware support from generators without hardware dedicated to generating entropy.
4. List the advantages and disadvantages of memory protection at the ISA level.
5. Describe key design issues of a trusted execution environment (TEE) to support virtual machines.

AR-Quantum: Quantum Architectures

KA Core:

1. Principles (See also: AL-Models: 8)
   a. The wave-particle duality principle
   b. The uncertainty principle in the double-slit experiment
   c. What is a Qubit? Superposition, interference, and measurement. Photons as qubits
   d. Systems of two qubits, Entanglement, Bell states, The No-Signaling theorem
2. Axioms of QM: superposition principle, measurement axiom, unitary evolution
3. Single qubit gates for the circuit model of quantum computation: X, Z, H
4. Two qubit gates and tensor products, working with matrices
5. The No-Cloning Theorem. The Quantum Teleportation protocol
6. Algorithms (See also: AL-Foundational)
   a. Simple quantum algorithms: Bernstein-Vazirani, Simon's algorithm
   b. Implementing Deutsch-Josza with Mach-Zehnder Interferometers
   c. Quantum factoring (Shor's Algorithm)
   d. Quantum search (Grover's Algorithm)
7. Implementation aspects (See also: SPD-Interactive)
   a. The physical implementation of qubits
   b. Classical control of a Quantum Processing Unit (QPU)
   c. Error mitigation and control, NISQ and beyond

   d. Measurement approaches
8. Emerging Applications
   a. Post-quantum encryption
   b. The Quantum Internet
   c. Adiabatic quantum computation (AQC) and quantum annealing

Illustrative Learning Outcomes:
KA Core:
1. Discuss how a quantum object produced as a particle propagates like a wave and is detected as a particle with a probability distribution corresponding to the wave.
2. Discuss the quantum-level nature that is inherently probabilistic.
3. Express your view on entanglement that can be used to create non-classical correlations, but there is no way to use quantum entanglement to send messages faster than the speed of light.
4. Describe quantum parallelism and the role of constructive vs destructive interference in quantum algorithms given the probabilistic nature of measurement(s).
5. Analyze a code snippet providing the role of quantum Fourier transform (QFT) in Shor's algorithm.
6. Write a program to implement Shor's algorithm in a simulator, highlighting the classical components and aspects of Shor's algorithm.
7. Enumerate the specifics of each qubit modality (e.g., trapped ion, superconducting, silicon spin, photonic, quantum dot, neutral atom, topological, color center, electron-on-helium).
8. Contrast AQC with the gate model of quantum computation and the problems each is better suited to solve.

AR-SEP: Sustainability Issues

Non-core:
1. Environmental impacts of implementation decisions
   a. Sustainability goals, resource consumption, and economic viability
   b. Carbon footprint, hardware electronic waste
   c. The energy footprint of data centers at various workloads (e.g., AI model training and use)
   d. Guidelines for sustainable design standards

Illustrative Learning Outcomes:
Non-core:
1. Assess the environmental impacts of a given project's deployment (e.g., the energy consumption of CPUs and GPUs, contribution to e-waste, and effect of hardware virtualization in data centers).

Professional Dispositions

- **Self-directed:** Students should increasingly become self-motivated to acquire complementary knowledge.
- **Proactive:** Students should exercise control and anticipate issues related to the underlying computer system.